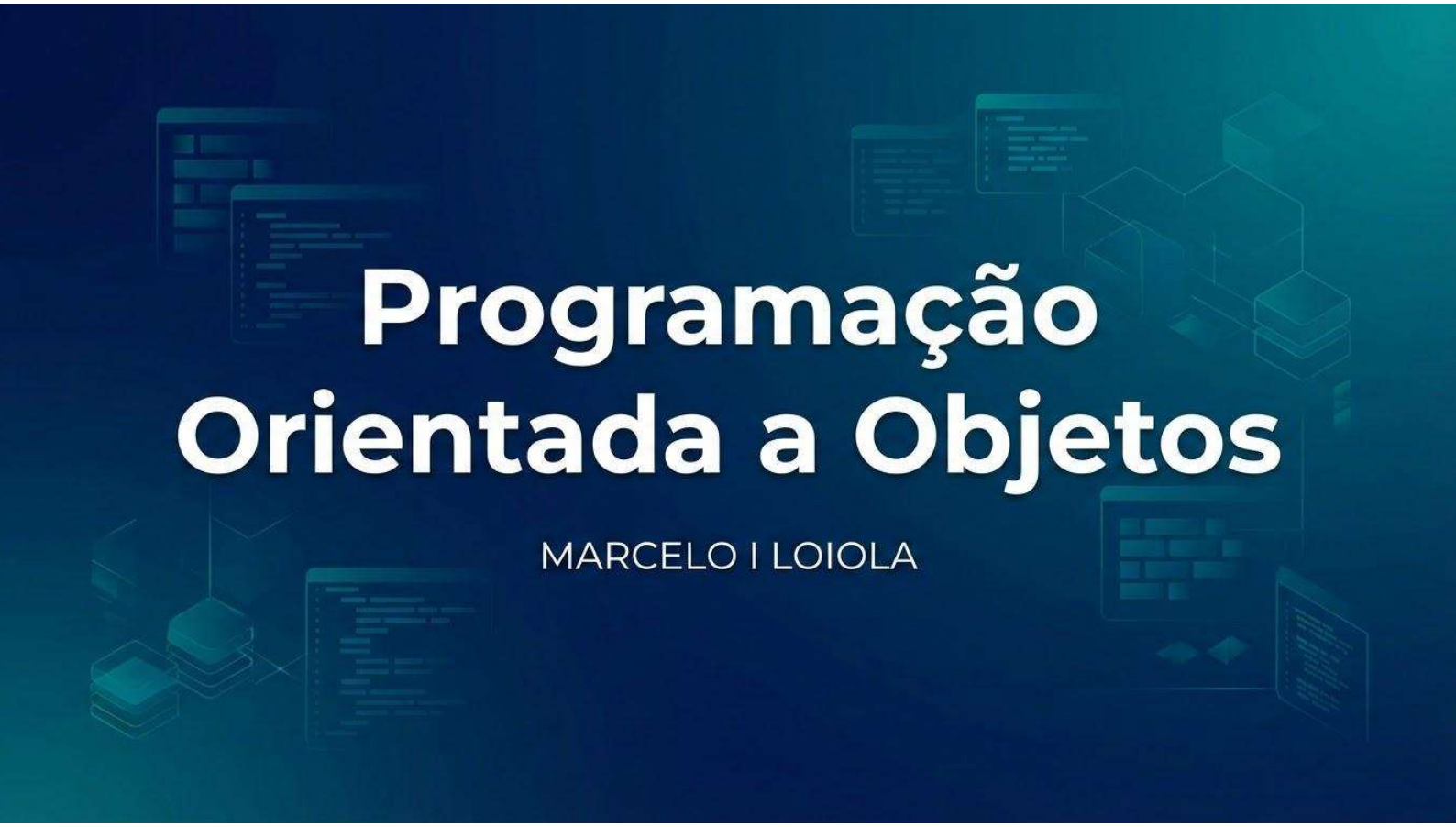


O Caos da Era Procedural

- No começo, a oficina de Hefesto era uma bagunça. Quando o Rei pedia 50 espadas, Hefesto pegava um pedaço de ferro, batia, esquentava e afiava. Depois pegava outro, batia, esquentava e afiava.
- Se o Rei dissesse: "Hefesto, agora quero que todas as espadas tenham o cabo vermelho!", o pobre ferreiro tinha que correr pela oficina, encontrar cada uma das 50 espadas espalhadas e pintar uma por uma. Era o caos.

O Caos da Era Procedural

- O código — digo, a oficina — era difícil de manter, desorganizado e qualquer mudança causava um colapso. Isso, era o que sofríamos na Programação Procedural ou estruturada pura.
- Cansado de sofrer, Hefesto teve uma ideia genial. Ele não ia mais fazer espadas do zero. Ele criou um **Molde Mestre**.

O Caos da Era Procedural

- Este molde não era uma espada. Era apenas a ideia, a definição, o projeto de uma espada. No molde estava escrito:
  - **Atributos** (Características): Toda espada feita aqui terá um tamanho, uma cor de cabo e um nível de afiação.
  - **Métodos** (Ações): Toda espada feita aqui poderá "cortar" e "defender".

O Caos da Era Procedural

- Na computação, chamamos esse "**Molde Mestre**" de **Classe**. Ela é o plano arquitetônico. Ela não existe na memória RAM como um dado vivo ainda, ela é apenas a definição.
- Com o molde pronto, Hefesto começou a produção. Ele despejava metal no molde e pronto! Nascia uma espada real. Despejava de novo, pronto,outra espada.

O Caos da Era Procedural

- Cada espada **criada a partir do molde** é o que chamamos de **Objeto** (ou Instância).
  - A "Espada A" tem o cabo azul.
  - A "Espada B" tem o cabo vermelho. Elas são independentes. Se a "Espada A" quebrar (mudar seu estado), a "Espada B" continua intacta. Mas ambas nasceram da mesma **Classe**.

JAVA

- Quais eram os seus maiores problemas quando se programava na década de 1990?
- *Ponteiros?* Gerenciamento de memória? *Organização?* Falta de bibliotecas? *Ter de reescrever parte do código ao mudar de sistema operacional?* Custo financeiro de usar a tecnologia?
- A linguagem Java resolve bem esses problemas que, até então, apareciam com frequência nas outras linguagens.

JAVA

- Motivação para a criação da plataforma Java era de que essa linguagem fosse usada em pequenos dispositivos. Mas teve seu lançamento focado no uso em clientes web (browsers) para rodar pequenas aplicações(applets).
- Hoje em dia, Java ganhou destaque no lado do servidor.

MÁQUINA VIRTUAL

Independência de Sistema Operacional

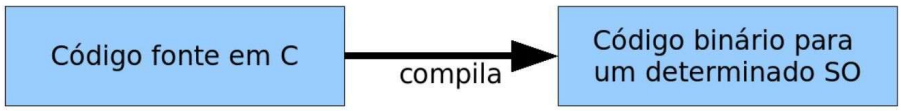
O Java utiliza o conceito de máquina virtual, uma camada extra entre a aplicação e o sistema operacional. Ela é responsável por traduzir as chamadas da aplicação para o sistema operacional em que está rodando. A maneira de abrir uma janela é a mesma, independentemente de ser Linux ou Windows.

JAVA

- O Java foi criado pela antiga Sun Microsystems e mantido por meio de um comitê (http://www.jcp.org). Com a compra da Sun pela Oracle em 2009, A página principal do Java é: http://www.oracle.com/technetwork/java/
- Diversos grupos de usuários se formaram para tentar disseminar o conhecimento da linguagem. GUJ (http://www.guj.com.br), uma comunidade virtual com artigos, tutoriais e fórum para tirar dúvidas, este sendo o maior em língua portuguesa.

MÁQUINA VIRTUAL

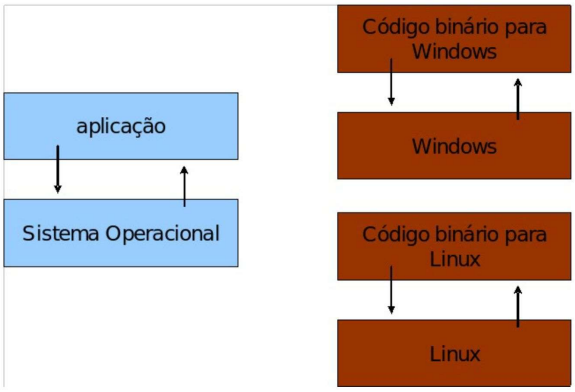
- Em uma linguagem de programação como C e Pascal, temos a seguinte situação quando vamos compilar um programa:



- O código fonte é compilado para código de máquina específico de uma plataforma e sistema operacional. Muitas vezes, o próprio código fonte é desenvolvido visando uma única plataforma!

MÁQUINA VIRTUAL

- Esse código executável (binário) resultante será exercido pelo sistema operacional e, por esse motivo, ele deve saber conversar com o sistema operacional em questão.

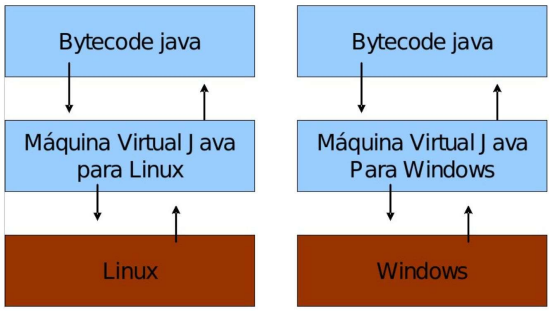


MÁQUINA VIRTUAL

- Isto é, temos um código executável para cada sistema operacional. É necessário compilar uma vez para Windows, outra para o Linux e assim por diante caso queiramos que esse nosso software seja utilizado em várias plataformas. Esse é o cenário de aplicativos como o OpenOffice, Firefox e outros.
- Na maioria das vezes, a sua aplicação se vale das bibliotecas do sistema operacional, por exemplo, a de interface gráfica para desenhar as telas. A biblioteca de interface gráfica do Windows é bem diferente das do Linux:

MÁQUINA VIRTUAL

- Já o Java utiliza o conceito de máquina virtual, no qual existe, entre o sistema operacional e a aplicação, uma camada extra responsável por traduzir – mas não apenas isso – o que sua aplicação deseja fazer para as respectivas chamadas do sistema operacional em que ela está rodando no momento:



MÁQUINA VIRTUAL

- A maneira em que se abre uma janela no Linux ou no Windows é a mesma: você ganha independência de sistema operacional. Não é preciso se preocupar em qual sistema operacional sua aplicação está rodando nem em que tipo de máquina, configurações, etc.
- Uma máquina virtual é um conceito bem mais amplo que o de um interpretador. É como um computador de mentira: apresenta tudo o que um computador tem. Ela é responsável por gerenciar memória, Threads, a pilha de execução, etc.

MÁQUINA VIRTUAL

- Sua aplicação roda sem nenhum envolvimento com o sistema operacional, sempre conversando apenas com a Java Virtual Machine (JVM).
- Como tudo passa pela JVM, ela pode tirar métricas, decidir em qual lugar é melhor alocar a memória, além de isolar totalmente a aplicação do sistema operacional.

MÁQUINA VIRTUAL

- Se uma Java Virtual Machine termina abruptamente, só as aplicações que estavam rodando nela irão terminar: isso não afetará outras JVMs que estejam rodando no mesmo computador nem o sistema operacional.

- Essa camada de isolamento também é interessante quando pensamos em um servidor que não pode se sujeitar a rodar código que possa interferir a boa execução de outras aplicações.

MÁQUINA VIRTUAL

- A máquina virtual, não entende código Java, mas compreende um código de máquina específico. Esse códigoé gerado por um compilador Java, como o javac, e é conhecido por "bytecode". O compilador Java gera esse bytecode que, diferente das linguagens sem máquina virtual, servirá para diferentes sistemas operacionais, já que ele será traduzido pela JVM.

WRITE ONCE, RUN ANYWHERE

Esse era um slogan que a Sun usava para o Java, já que você não precisa reescrever partes da sua aplicação toda vez que quiser mudar de sistema operacional.

JAVA LENTO? HOTSPOT E JIT

- *Hotspot* é a tecnologia que a JVM utiliza para detectar *pontos quentes* da sua aplicação: código que é executado muito provavelmente dentro de um ou mais loops.
- Quando a JVM julgar necessário, ela vai compilar esses códigos para instruções realmente nativas da plataforma, tendo em vista que isso irá provavelmente melhorar a performance da sua aplicação. Esse compilador é o *JIT: Just inTime Compiler*, que aparece bem na hora em que precisa.

JAVA LENTO? HOTSPOT E JIT

- Por que a JVM não compila tudo antes de executar a aplicação? É que, teoricamente, compilar de forma dinâmica, na medida do necessário, pode gerar uma performance melhor.
- O motivo é simples: imagine um .exe gerado pelo VisualBasic, gcc ou Delphi. Ele é estático e já foi otimizado com base em heurísticas. O compilador pode ter tomado uma decisão não tão boa.

JAVA LENTO? HOTSPOT E JIT

- Já a JVM, por estar compilando dinamicamente durante a execução, pode perceber que um determinado código não está com a performance adequada e otimizar mais um pouco aquele trecho ou ainda mudar a estratégia de otimização.
- É por esse motivo que as JVMs mais recentes, em alguns casos, chegam a ganhar de códigos C compilados com o GCC 3.x.

JVM? JRE? JDK?

- **JVM**: apenas a virtual machine. Esse download não existe, pois ela sempre vem acompanhada.
- **JRE**: Java Runtime Environment. Ambiente de execução Java, formado pela JVM e bibliotecas, tudo que você precisa para executar uma aplicação Java. Mas precisamos de mais.
- **JDK**: Java Development Kit. Nós, desenvolvedores, faremos o download do JDK do Java SE(Standard Edition). Ele é formado pela JRE somado às ferramentas como o compilador.

ONDE USAR, E OS OBJETIVOS DO JAVA

- Você pode achar que o Java tem menor produtividade quando comparado com a linguagem que está acostumado.
- É preciso ficar claro que a premissa do Java não é a de criar sistemas pequenos nos quais temos um ou dois desenvolvedores mais rapidamente que linguagens do tipo PHP, Python e outras.
- O foco da plataforma é outro: aplicações de *médio a grande porte*.

ONDE USAR, E OS OBJETIVOS DO JAVA

- Criar a primeira versão de uma aplicação usando Java, mesmo utilizando IDEs e ferramentas poderosas, será mais trabalhoso que muitas linguagens script ou de alta produtividade.
- Porém, com uma linguagem orientada a objetos e madura como o Java, será extremamente mais fácil e rápido fazer alterações no sistema desde que você siga as boas práticas e recomendações sobre *design* orientado a objetos.

ONDE USAR, E OS OBJETIVOS DO JAVA

- Além disso, a quantidade enorme de bibliotecas gratuitas para realizar os mais diversos trabalhos é um ponto fortíssimo para adoção do Java: você pode criar uma aplicação sofisticada usando diversos recursos sem precisar comprar um componente específico, que costuma ser caro.

ONDE USAR, E OS OBJETIVOS DO JAVA

- Cada linguagem tem seu espaço e seu melhor uso. A utilização do Java é interessante em aplicações que crescerão, nas quais a legibilidade do código é importante e temos muita conectividade, além de serem compatíveis com muitas plataformas (ambientes e sistemas operacionais) heterogêneas (Linux, Unix, OSX e Windows, misturados).
- Existe uma grande quantidade de ofertas de emprego procurando desenvolvedores Java para trabalhar com sistemas web e aplicações de integração no servidor.

COMPILANDO O PRIMEIRO PROGRAMA

- Vamos para o nosso primeiro código! O programa que imprime uma linha simples. Para mostrar uma linha, podemos fazer:

```
System.out.println("Minha primeira aplicação Java!");
```

- Mas esse código não será aceito pelo compilador Java. O Java é uma linguagem bastante burocrática e precisa de mais do que isso para iniciar uma execução. Veremos os detalhes e os porquês durante a disciplina.

```
class MeuPrograma {
    public static void main(String[] args) {
        System.out.println("Minha primeira aplicação Java!");
    }
}
```

COMPILANDO O PRIMEIRO PROGRAMA

- Após digitar o código acima, grave-o como MeuPrograma.java em algum diretório. A fim de compilar, você deve pedir para que o compilador de Java da Oracle, chamado javac , gere o bytecode correspondente ao seu código Java.
- Depois de compilar, o bytecode foi gerado. Quando o sistema operacional listar os arquivos contidos no diretório atual, você verá que um arquivo .class foi gerado com o mesmo nome da sua classe Java.